MAARS: Technical Architecture and Design Specification

From OpenClaw to the Sovereign Civilization Layer

Project Codename: MAARS (Master Autonomous Agentic Runtime System) **Classification:** Internal Technical Specification — Development Team Reference **Version:** 1.0 **Date:** March 21, 2026

Table of Contents

1. Executive Summary & Strategic Context
2. The Evolutionary Trajectory: Understanding the Four Phases
3. Architectural Philosophy & Core Design Principles
4. Phase 1: OpenClaw — The Personal Assistant (Baseline)
5. Phase 2: ApexClaw / Master ABU Vessel — Enterprise AgenticOps
6. Phase 3: The Autonomous Machine Economy — Agent-to-Agent Commerce
7. Phase 4: MAARS — The Sovereign Civilization Layer
8. The MAARS Engine: Deep Technical Specification
9. The MAARS Vision Layer: Deep Technical Specification
10. Master Data Model & API Surface
11. Technology Stack Reference
12. Implementation Roadmap
13. Security, Compliance & Patentability
14. Competitive Landscape & Market Context
15. References

---

1. Executive Summary & Strategic Context

The transition from conversational AI to agentic AI represents the most fundamental shift in computing since the advent of the internet. OpenClaw emerged as the dominant open-source framework in this space, rapidly becoming the “Linux of agentic AI” — a phrase coined by NVIDIA CEO Jensen Huang at GTC 2026 [1]. However, despite its massive adoption (over 13,000 community-built skills on ClawHub), OpenClaw suffers from critical architectural flaws, security vulnerabilities, and a steep learning curve that fundamentally limits its enterprise viability [2] [3]. NVIDIA’s NemoClaw attempted to address the security gaps by wrapping OpenClaw in policy guardrails, but it failed to solve the core issues of deterministic execution, multi-agent orchestration, and non-technical usability [4].

MAARS (Master Autonomous Agentic Runtime System) is the definitive answer to the enterprise market’s unmet need. It is not a patch on top of OpenClaw, nor a security wrapper like NemoClaw. MAARS is a ground-up, enterprise-grade Software-as-a-Service (SaaS) platform designed to orchestrate, secure, and monetize Autonomous Business Units (ABUs) at scale. The platform synthesizes the most advanced capabilities from twelve leading open-source projects and commercial platforms — including DeerFlow, Mem0, Cognee, Agency-Agents, PentAGI, MiroFish, AutoRA, Heretic, Portkey, Arize Phoenix, and Strata Maverics — into a single, cohesive architecture.

The global agentic AI market is projected to grow from $9.14 billion in 2026 to$139.19 billion by 2034, representing a 43.84% CAGR [5]. Gartner projects agentic AI spending alone will reach $201.9 billion in 2026, overtaking chatbot spending by 2027 [6]. MAARS is positioned to capture a significant share of this market by solving the exact pain points that OpenClaw and NemoClaw leave unaddressed: security, enterprise compliance, multi-agent orchestration, and non-technical usability.

This document is the single authoritative technical reference for the MAARS development team. It covers every architectural decision, technology choice, data model, API surface, and implementation detail required to build the system to 100% completion. The development team should treat this document as the ground truth for all engineering decisions.

---

2. The Evolutionary Trajectory: Understanding the Four Phases

To understand what MAARS is and why it is built the way it is, the development team must first understand the four-phase evolutionary trajectory that leads to it. Each phase represents a distinct paradigm shift in how AI agents relate to the world.

2.1 Phase 1: OpenClaw — The Agent as a Tool

OpenClaw (formerly Clawdbot/Moltbot) is an open-source, local-first agentic operating system. It acts as a gateway connecting LLMs to local file systems, shell environments, and messaging platforms [7]. In this paradigm, the agent is a **tool**. It lives on local hardware, automates personal tasks, and relies entirely on a human's credentials and authorization. It has no independent identity, no capital, and no ability to interact with other agents. It is a highly capable digital proxy for a single human.

2.2 Phase 2: ApexClaw / Master ABU Vessel — The Agent as an Employee

Phase 2 scales the agent to the enterprise. It operates within strict corporate guardrails, utilizes WASM sandboxing, and connects to enterprise systems via Agent-Native APIs. In this paradigm, the agent is an **employee**. It acts on behalf of a corporation but still requires human-in-the-loop approval for high-stakes actions. It has a managed identity, operates within defined liability caps, and is subject to continuous observability and audit.

2.3 Phase 3: The Autonomous Machine Economy — The Agent as an Economic Actor

Phase 3 marks the leap from "employee" to "economic actor." Agents no longer just execute tasks within a single organization; they transact with other agents across organizational boundaries. They hold their own capital (ERC-4337 Smart Accounts), negotiate their own contracts, and purchase their own resources using the x402 machine-to-machine payment protocol. The platform evolves from an orchestration engine into an Agentic Marketplace and Clearinghouse.

2.4 Phase 4: MAARS — The Agent as a Sovereign Entity

Phase 4 is the ultimate realization of the agentic trajectory. Agents become **sovereign entities** with persistent identity, legal standing, and self-determined goals. They have W3C Decentralized Identifiers (DIDs) anchored to blockchains, legally recognized in forward-thinking jurisdictions (e.g., Wyoming DAO LLCs) as corporate entities. They form Decentralized Autonomous Organizations (DAOs) to govern themselves. They run on Decentralized Physical Infrastructure Networks (DePIN), making them impossible to shut down by any single government or corporation. MAARS becomes the foundational operating system for a new kind of civilization.

---

3. Architectural Philosophy & Core Design Principles

The MAARS architecture is built upon a set of non-negotiable design principles. Every engineering decision must be evaluated against these principles.

3.1 Hierarchical Agent Orchestration

MAARS prioritizes a hierarchical agent structure with multiple levels of nesting: Master Agents, Sub-Agents, and Sub-Sub-Agents. A Master Agent receives a high-level goal from a human operator, decomposes it into a Directed Acyclic Graph (DAG) of sub-tasks, and dispatches each sub-task to a specialized Sub-Agent. Sub-Agents may themselves spawn further specialized agents for narrow tasks. This hierarchical structure enables organized, multi-level task decomposition and prevents the "agent sprawl" problem that plagues flat architectures.

The system must support **recursive agent spawning** — the ability for any agent in the hierarchy to spawn new agents as needed. It must also include a **right-sizing mechanism** that assigns the appropriate cognitive tier (Nano, Mid, or Frontier) to each sub-task based on its complexity and cost requirements.

3.2 Deterministic Guardrails Over Probabilistic Reasoning

OpenClaw's fatal flaw is that it relies on LLM reasoning to make security and compliance decisions. This is fundamentally non-deterministic and cannot be used in regulated industries. MAARS enforces a strict separation between **probabilistic reasoning** (the LLM's domain) and **deterministic execution** (the rules engine's domain). Business rules are hard-coded in the `vessel-economics` module. If an agent decides to execute a trade, send a patient record, or sign a contract, it must pass through a deterministic, human-in-the-loop approval gate — not a prompt.

3.3 Zero Trust for Non-Human Identities (NHI)

Every agent in MAARS is treated as an untrusted entity by default. Before any agent is assigned a task, `vessel-identity` provisions a Just-In-Time (JIT) token that is cryptographically bound to the specific task, time window, and permission scope. This token cannot be reused for other tasks. The Continuous Access Evaluation Protocol (CAEP) engine monitors agent behavior in real-time and can revoke tokens instantly if anomalous behavior is detected.

3.4 WASM-First Sandboxing

Every agent and skill in MAARS runs in an isolated WebAssembly (WASM) container. Unlike Docker, which isolates at the OS level, WASM provides a memory-safe, sandboxed execution environment with millisecond startup times and granular, capability-based permissions. An agent cannot access the host OS, network, or file system unless explicitly granted permission for that specific task. This eliminates OpenClaw's "Security Nightmare" at the architectural level.

3.5 User-Configurable LLM Integration

MAARS must never hardcode LLM choices. The `vessel-llm-router` provides a Portkey-inspired gateway that abstracts all LLM providers. Users can bring their own API keys for any supported provider (OpenAI, Anthropic, Google, Mistral, etc.) or connect custom/self-hosted models. The system routes requests to the optimal model based on cost, latency, and capability requirements, with automatic fallback chains if a provider is unavailable.

3.6 Event-Driven, Microservices Architecture

MAARS is decomposed into twelve specialized microservices that communicate asynchronously via Apache Kafka and synchronously via gRPC/REST. This event-driven architecture ensures high throughput, loose coupling, and independent scalability of each service. It also provides a natural audit trail, as every event on the Kafka bus is a durable, replayable record.

3.7 Multi-Tenant Data Isolation

MAARS is a multi-tenant SaaS platform. Data isolation between tenants is non-negotiable. The platform employs a Hybrid Tenancy Model: shared Kubernetes clusters with namespace isolation for stateless compute, and separate logical databases/keyspaces in Neo4j and Vector DBs per tenant for stateful memory storage.

---

4. Phase 1: OpenClaw — The Personal Assistant (Baseline)

This section provides the development team with a thorough understanding of OpenClaw's architecture, strengths, and weaknesses. Understanding the baseline is essential for understanding every architectural decision in MAARS.

4.1 OpenClaw Architecture Overview

OpenClaw operates on a **hub-and-spoke architecture** centered around a single Gateway. The Gateway is a locally running WebSocket server that acts as the control plane, brokering communication between chat interfaces, the AI model, and local tools [8].

Component	Description
Gateway	A locally running WebSocket server. The single point of control for all agent activity.
Channel Adapters	Plugins that allow the agent to receive commands from WhatsApp, Telegram, Slack, iMessage, and Discord.
LLM Connector	Connects to any LLM (Claude, GPT, Gemini, local models) via API.
Tool Executor	Executes tools directly on the host machine: file system, shell, browser, calendar.
Memory Store	Local SQLite database and Markdown files (`MEMORY.md`) for persistent context.
Skill Ecosystem	Over 13,000 community-built "skills" on ClawHub that extend the agent's capabilities.

The execution flow is simple: a user sends a message via a chat interface, the Gateway receives it, passes it to the LLM with the current memory context, the LLM decides which tool to call, and the tool executes directly on the host machine.

4.2 OpenClaw Strengths

OpenClaw's viral adoption is not accidental. It has genuine strengths that MAARS must preserve or improve upon.

Its **ecosystem** is its most significant asset. Over 13,000 community-built skills represent an enormous amount of collective intelligence and tool coverage. Its **model agnosticism** allows users to plug in any LLM, giving them flexibility and cost control. Its **channel integration** — living inside WhatsApp, Telegram, and Slack — means the agent operates where users already work, dramatically lowering the adoption barrier. Its **local-first execution** gives users full control over their data and eliminates cloud dependency for personal tasks.

4.3 OpenClaw Critical Weaknesses & The Enterprise Gap

OpenClaw's architecture creates a fundamental "Context Gap" when scaled to enterprise environments [9]. These are not minor issues; they are architectural impossibilities that cannot be patched.

Security Architecture Failure OpenClaw agents run with host-level privileges. A compromised agent — via prompt injection or a malicious skill — can exfiltrate data, delete files, or access sensitive credentials [2] [7]. Meta and other major tech firms have banned OpenClaw usage internally due to these security concerns [10]. The skill supply chain is unverified; any of the 13,000+ skills on ClawHub could be malicious. NemoClaw's policy guardrails do not solve this because they wrap the agent at the network level, not the execution level.

Flat Memory Architecture OpenClaw stores memory in local SQLite databases and Markdown files. If Agent A updates a customer record, Agent B on another machine cannot see it [9]. There is no shared context, no relationship graph, and no ability to reason about complex relationships over time. This makes it impossible to deploy multiple coordinated agents in an enterprise environment.

Non-Deterministic Execution Business logic is embedded in prompts rather than code. The LLM decides whether to send a patient record or execute a trade. This is fundamentally non-deterministic and cannot be used in regulated industries (Healthcare, Finance, Legal) where compliance requires 100% predictable behavior [3].

Token Waste & Inefficiency OpenClaw brute-forces web browsers to extract information, consuming enormous amounts of tokens on HTML parsing. For enterprise use cases, this is both expensive and unreliable. A structured API approach reduces token usage by up to 90%.

No Multi-Agent Orchestration OpenClaw has a single agent per instance. There is no concept of a Master Orchestrator spawning specialized sub-agents, no right-sizing engine, and no parallel execution. Complex enterprise tasks that require multiple specialized agents working in parallel are simply impossible.

Weakness	Impact	MAARS Solution
Host-level privilege execution	Data exfiltration, RCE via prompt injection	WASM sandboxing with capability-based permissions
Flat local memory	No shared context across agents/machines	`vessel-memory`: Mem0 + Cognee Neo4j GraphRAG
Non-deterministic execution	Compliance failure in regulated industries	Deterministic Policy Engine + Agentic Escrow
Token waste (browser scraping)	High cost, low reliability	Agent-Native APIs via `vessel-integrations`
No multi-agent orchestration	Cannot handle complex enterprise tasks	`vessel-orchestrator` + `vessel-swarm` hierarchy
No audit trail	Legal liability void	Cryptographic Merkle-tree audit log
No identity management	Insider threat, no access control	`vessel-identity`: JIT + ABAC + CAEP

---

5. Phase 2: ApexClaw / Master ABU Vessel — Enterprise AgenticOps

Phase 2 is the core of the MAARS platform. It transforms the personal assistant into an Enterprise AgenticOps Platform. The architecture is built around twelve specialized microservices organized into six layers.

5.1 The Six-Layer Architecture

The MAARS engine is organized into six distinct layers, each with a specific responsibility. Understanding this layering is critical for the development team, as it defines the boundaries of each service and the flow of data through the system.

Layer	Services	Responsibility
Layer 1: Orchestration & Governance	`vessel-gateway`, `vessel-orchestrator`, `vessel-swarm`	Ingestion, task planning, agent lifecycle management
Layer 1.5: Identity, Trust & Federation	`vessel-identity` (JIT, ABAC, CAEP, DID, A2A, ANS, AIM)	Zero Trust NHI, cross-org trust, cryptographic identity
Layer 2: Economic & Legal	`vessel-economics` (Ledger, Escrow, Compliance)	Financial transactions, liability enforcement, audit
Layer 3: Reasoning & Memory	`vessel-llm-router`, `vessel-memory`	LLM routing, persistent context, knowledge graph
Layer 3.5: Observability & Guardrails	`vessel-observability` (OpenTelemetry, Guardrail, Context Graph, Insurance API)	Real-time monitoring, failure prevention, telemetry
Layer 4: Execution & Simulation	`vessel-sandbox`, `vessel-simulation`, `vessel-research`, `vessel-self-improvement`	Secure execution, digital twin, autonomous research, self-optimization

5.2 Layer 1: Orchestration & Governance

5.2.1 `vessel-gateway`

The `vessel-gateway` is the single ingress point for all external traffic. It is built on Nginx and Kong API Gateway. Its responsibilities are:

Prompt Injection Filtering: All incoming prompts are scanned by an ASI01-inspired filter before being passed to the orchestrator. The filter detects and blocks known prompt injection patterns (e.g., “Ignore previous instructions”, “Act as DAN”). Suspicious prompts are flagged and logged.

WebSocket Management: The gateway maintains persistent WebSocket connections with the frontend (`vessel-interface`) for real-time event streaming. It manages connection lifecycle, heartbeats, and reconnection logic.

Rate Limiting & Tenant Routing: Each tenant is assigned a rate limit based on their plan tier. The gateway enforces these limits using Redis-backed counters and routes requests to the correct tenant namespace in the Kubernetes cluster.

Authentication Middleware: The gateway validates JWT tokens issued by `vessel-identity` before forwarding requests to internal services. It also handles OAuth 2.0 flows for initial authentication.

Implementation Notes:

- Language: Go (for high-throughput, low-latency performance)
- Kong plugins: `rate-limiting`, `jwt`, `request-transformer`, `response-transformer`
- Redis: Used for rate limiting counters and session caching
- Health checks: `/health` endpoint for Kubernetes liveness/readiness probes

5.2.2 `vessel-orchestrator`

The `vessel-orchestrator` is the “brain” of the MAARS engine. It is responsible for decomposing high-level goals into executable task graphs and managing the execution lifecycle of those tasks.

LangGraph DAG Planner: When a `GoalPacket` is received, the orchestrator uses a LangGraph-inspired Directed Acyclic Graph (DAG) planner to decompose the goal into a `TaskGraph`. Each node in the graph is a `TaskDefinition` with a specific instruction, tool allowlist, budget ceiling, and assigned agent tier. The DAG captures dependencies between tasks, ensuring that tasks are executed in the correct order and that parallel tasks are executed concurrently.

Right-Sizing Engine: Before dispatching tasks, the orchestrator’s Right-Sizing Engine analyzes each task’s complexity and assigns the appropriate cognitive tier:

- **Nano-Agents** (e.g., Llama 3 8B): Simple, deterministic tasks — data formatting, file operations, simple lookups.
- **Mid-Agents** (e.g., Mistral 7B, Llama 3 70B): Moderate complexity — summarization, classification, structured data extraction.
- **Frontier-Agents** (e.g., GPT-4.1, Claude 3.5, Gemini 2.5): High complexity — multi-step reasoning, code generation, complex analysis.

Circuit Breaker: The orchestrator implements a circuit breaker pattern. If a sub-task fails repeatedly, the circuit breaker opens, preventing further attempts and escalating the failure to the human operator via the Escrow & Guardrail Inbox.

Implementation Notes:

- Language: Python (FastAPI)
- Orchestration Framework: LangGraph (for stateful, graph-based workflow management)
- State Store: PostgreSQL (for `GoalPacket`, `TaskGraph`, `TaskDefinition` persistence)
- Event Bus: Kafka (for publishing `goal.created`, `task.assigned`, `task.completed` events)

5.2.3 `vessel-swarm`

The `vessel-swarm` is the agent registry and lifecycle manager. It maintains pools of pre-warmed agents at each cognitive tier and handles the spawning, assignment, and decommissioning of agents.

Agent Registry: Each agent is registered as an `AgentProfile` with a unique `agent_id`, capabilities list, model tier, system prompt version, and performance score. The registry is the source of truth for all active agents.

Hierarchical Spawning: When the orchestrator dispatches a task, `vessel-swarm` selects the appropriate agent from the pool or spawns a new one. Agents can themselves request the spawning of sub-agents by sending a `spawn_sub_agent` event to the Kafka bus, which `vessel-swarm` handles.

Lifecycle Management: Agents transition through states: `IDLE → BUSY → TERMINATED`. The swarm monitors agent health and automatically terminates agents that exceed their time or budget limits.

Implementation Notes:

- Language: Python (FastAPI)
- Agent Tiers: Nano (Llama 3 8B), Mid (Llama 3 70B / Mistral), Frontier (GPT-4.1 / Claude 3.5)
- Pre-warming: A configurable number of agents at each tier are kept warm to minimize cold-start latency
- Kafka Topics: `agent.spawned`, `agent.idle`, `agent.terminated`

5.3 Layer 1.5: Identity, Trust & Federation

The identity layer is the most complex and most critical component of the MAARS security model. It implements Zero Trust for Non-Human Identities (NHI) and provides the cryptographic foundation for cross-organizational trust in Phase 3 and Phase 4.

5.3.1 Just-In-Time (JIT) Identity Provisioner

When the orchestrator dispatches a task, `vessel-identity` provisions a JIT token for the assigned agent. This token is:

- **Task-Scoped:** Valid only for the specific `task_id` it was issued for.
- **Time-Bounded:** Expires at the task's deadline or after a configurable maximum duration.
- **Permission-Scoped:** Contains only the permissions required for the task's `tool_allowlist`.
- **Cryptographically Bound:** Signed with the platform's private key and verifiable by any service.

The JIT token is stored in the `AgentIdentity` table with its full `scope_json` and `delegation_chain_json`. This creates a complete, auditable record of every permission ever granted to every agent.

5.3.2 ABAC Policy Engine

The Attribute-Based Access Control (ABAC) engine evaluates access requests based on attributes of the agent (tier, capabilities), the resource (sensitivity level, data classification), and the environment (time of day, tenant policy). This provides far more granular control than role-based access control (RBAC).

The `DelegationChain` entity records the human delegator, the purpose of the delegation, and the `liability_cap_usd` — the maximum financial liability the human is authorizing the agent to incur. This is the foundation of the Agentic Escrow Engine.

5.3.3 CAEP Engine (Continuous Access Evaluation Protocol)

The CAEP engine monitors agent behavior in real-time and can revoke JIT tokens instantly if anomalous behavior is detected. Triggers for revocation include:

- Agent attempting to access resources outside its `scope_json`
- Agent exhibiting behavior patterns matching known attack signatures
- Human operator manually revoking access via the Trust Center UI
- `vessel-observability` flagging a reasoning chain as suspicious

5.3.4 W3C DID + VC Engine (Phase ¾ Prerequisite)

For cross-organizational trust (Phase 3) and legal personhood (Phase 4), every agent is issued a W3C Decentralized Identifier (DID) and a Verifiable Credential (VC). The DID is generated using

the `did:key` method with Ed25519 keypairs. The VC contains the agent's capabilities, liability cap, and issuing organization.

This DID/VC pair is published as an Agent Card at a well-known URL (e.g., `https://agent-a.orgA.maars.ai/.well-known/agent.json`) and registered in the Agent Name Service (ANS) registry with a Merkle tree transparency log entry.

5.3.5 A2A Protocol Server

The A2A Protocol Server implements the Agent2Agent (A2A) protocol specification (v1.0.0, donated to the Linux Foundation in June 2025) [11]. It exposes a JSON-RPC 2.0 over HTTPS endpoint that allows external agents to discover, negotiate with, and delegate tasks to MAARS agents.

Key A2A operations implemented:

- `message/send`: Send a task request to a MAARS agent.
- `tasks/get`: Check the status of a running task.
- `tasks/cancel`: Cancel a running task.
- `tasks/subscribe`: Subscribe to real-time task updates via Server-Sent Events (SSE).

5.3.6 ANS Registry Client

The Agent Name Service (ANS) Registry Client registers MAARS agents in the global ANS registry using FQDN-anchored identities (e.g., `agent-a.orgA.maars.ai`). This allows external organizations to discover MAARS agents by capability using a DNS-like lookup.

5.3.7 AIM — Non-Human Identity Governance

The AIM (Agent Identity Management) module provides append-only audit logging of all identity events (provisioning, revocation, delegation). It uses Ed25519 keypairs for signing and OAuth 2.0 token endpoints for external authentication.

5.4 Layer 2: Economic & Legal

5.4.1 On-Chain Treasury & Ledger

The `vessel-economics` module maintains an on-chain treasury for tokenized compute credits. Every `ToolExecution`, `LLMCall`, and `ExternalAPI` call is recorded as a `Transaction` with its cost in USD and an on-chain hash for auditability.

Monetization Model: MAARS operates on a Consumption-Based + Outcome-Based Hybrid Pricing Model:

- **Compute Credits:** Customers purchase credits burned based on agent tier and sandbox duration.
- **Outcome Bounties:** Fixed fees for successful high-value task completions (e.g., vulnerability discovery, research synthesis).
- **Escrow Fees:** A percentage fee on transactions processed through the Agentic Escrow Engine.
- **Insurance Telemetry API Access:** Premium tier access for underwriting partners to consume anonymized actuarial data.

5.4.2 Agentic Escrow Engine

The Agentic Escrow Engine is one of the most critical and novel components of MAARS. It is the mechanism that makes the platform safe for regulated industries.

When an agent attempts a high-stakes action (e.g., signing a contract, sending a patient record, executing a financial transaction), `vessel-economics` intercepts the action before it is committed. The action is held in a `EscrowAction` record with status `HELD`. The rules engine then evaluates the action against the human delegator's predefined `DelegationChain`:

- If the action's estimated cost is within the `liability_cap_usd`, and all policy rules pass, the action is automatically approved (`APPROVED`).
- If the action exceeds the liability cap or fails a policy rule, it is escalated to the human operator via the Escrow & Guardrail Inbox with status `ESCALATED`.
- If the human approves, the action is released (`APPROVED`). If rejected, it is cancelled (`REJECTED`).

Every escrow decision is recorded in the `ComplianceAudit` table with the full `rules_engine_decision_json` for SOC2/HIPAA compliance.

Implementation Notes:

- Smart contracts are implemented on a permissioned blockchain (e.g., Hyperledger Fabric for enterprise, or Ethereum L2 for Phase 3+).
- The escrow engine is a Python FastAPI service with a PostgreSQL backend.
- Kafka events: `escrow.held`, `escrow.approved`, `escrow.rejected`

5.4.3 Compliance Auditor

The Compliance Auditor is a PentAGI-inspired module that performs automated compliance checks against configurable policy frameworks (SOC2, HIPAA, GDPR, ISO 27001). It generates `ComplianceAudit` records for every escrow action and can trigger alerts for policy violations.

5.5 Layer 3: Reasoning & Memory

5.5.1 `vessel-llm-router`

The `vessel-llm-router` is a Portkey-inspired LLM-agnostic gateway. It abstracts all LLM providers and provides a unified interface for the rest of the system. Key features:

Multi-Provider Routing: Routes requests to the optimal LLM based on the task's cognitive tier, cost budget, and latency requirements. Supports OpenAI, Anthropic, Google, Mistral, Cohere, and any OpenAI-compatible API endpoint (for custom/self-hosted models).

Fallback Chains: If a primary provider is unavailable or returns an error, the router automatically falls back to a secondary provider. Fallback chains are configurable per tenant and per agent tier.

Semantic Prompt Cache: Caches LLM responses for semantically similar prompts using vector similarity search. This can reduce LLM costs by 30-60% for repetitive tasks.

Portable Prompt Compiler: Translates prompts from a universal meta-language into model-specific formats. This ensures that prompts designed for one model work correctly on another, preventing vendor lock-in.

User-Configurable API Keys: Tenants can bring their own API keys for any supported provider. The router stores these keys encrypted in the database and uses them for all requests from that tenant.

Implementation Notes:

- Language: Go (for high-throughput, low-latency routing)
- Supported providers: OpenAI (GPT-4.1, GPT-4.1-mini, o3), Anthropic (Claude 3.5, Claude 4), Google (Gemini 2.5 Flash, Gemini 2.5 Pro), Mistral, Cohere, custom OpenAI-compatible endpoints
- Semantic cache: Qdrant vector store with cosine similarity threshold of 0.95
- Kafka events: None (synchronous gRPC calls only)

5.5.2 `vessel-memory`

The `vessel-memory` module is the context engine for MAARS. It replaces OpenClaw's flat Markdown memory with a hierarchical, multi-tenant knowledge graph. It combines two complementary memory systems:

Mem0 Vector Store: Stores `MemoryNode` records as high-dimensional embeddings in Qdrant. Enables fast semantic search across an agent's memory. Supports three memory types:

- **Episodic Memory:** Specific events and interactions (e.g., “On March 15, the user asked me to schedule a meeting with John”).
- **Semantic Memory:** General facts and knowledge (e.g., “The user’s preferred meeting time is 10am”).
- **Procedural Memory:** How to perform tasks (e.g., “To book a flight, first check the user’s calendar, then search for flights”).

Cognee Neo4j Knowledge Graph: Stores `KnowledgeGraphNode` and `KnowledgeGraphEdge` records in a Neo4j graph database. Enables complex relationship queries (e.g., “What are all the contracts signed by agents in the last 30 days that involve Company X?”). GraphRAG retrieval combines graph traversal with vector search for highly contextual responses.

Memory Provenance Guard: Every `MemoryNode` has a `provenance_hash` that cryptographically links it to its source (e.g., a specific tool execution or LLM response). This prevents memory poisoning attacks and enables full auditability of the agent’s knowledge.

Privacy-Preserving Memory Federation (Phase 3+): Using FedE4RAG and memX principles, `vessel-memory` can share anonymized, policy-masked insights across tenants without exposing raw PII or GDPR-protected data. This enables cross-tenant learning while maintaining strict data isolation.

Implementation Notes:

- Vector Store: Qdrant (self-hosted on Kubernetes)
- Knowledge Graph: Neo4j (self-hosted on Kubernetes)
- Embedding Model: `text-embedding-3-large` (OpenAI) or `nomic-embed-text` (local)
- Tenant isolation: Separate Qdrant collections and Neo4j databases per tenant
- Memory expiry: Configurable TTL per memory type

5.6 Layer 3.5: Observability & Guardrails

5.6.1 OpenLLMetry Collector

The observability stack is built on OpenTelemetry with the OpenInference/OpenLLMetry extensions for AI-specific tracing. Every LLM call, tool execution, and agent decision is recorded as a `TraceSpan` with full context.

The collector aggregates spans into `ContextGraphSnapshot` records that capture the full reasoning chain for each task. These snapshots are stored in the `vessel-observability` database and are accessible via the `/app/canvas` UI for real-time inspection.

Metrics collected:

- Token consumption per agent, per task, per tenant
- LLM latency (time to first token, total generation time)
- Tool execution success/failure rates
- Guardrail intervention rates
- Escrow hold rates and approval/rejection ratios

5.6.2 Inline Guardrail Agent

The Inline Guardrail Agent is one of MAARS's key "Build Opportunity" components — it does not exist in any current open-source framework and represents a significant patentable innovation.

Before any tool execution is committed, the Guardrail Agent scores the agent's reasoning chain using a separate, isolated LLM judge. The judge evaluates:

- **Logic Coherence:** Does the proposed action logically follow from the task instructions?
- **Policy Compliance:** Does the proposed action violate any of the tenant's configured policies?
- **Anomaly Detection:** Does the proposed action match any known attack patterns (e.g., prompt injection, data exfiltration)?

The Guardrail Agent produces a `GuardrailDecision` with one of three outcomes:

- **PASS:** The action is safe to execute.
- **BLOCK:** The action is blocked and logged. The agent is instructed to retry with a different approach.
- **ESCALATE:** The action is suspicious and requires human review. An `InboxCard` is generated in the Escrow & Guardrail Inbox.

The Guardrail Agent feeds its decisions back into the `Context Graph Engine` (Arize Phoenix), which builds a `FailurePattern` database over time. These patterns are used to improve the guardrail's accuracy and to trigger `vessel-self-improvement` optimization runs.

5.6.3 Insurance Telemetry API

The Insurance Telemetry API is a novel monetization channel enabled by MAARS's comprehensive observability. It provides underwriting partners with anonymized, aggregated actuarial data about agent behavior:

- Guardrail intervention rates (proxy for agent risk)
- Escrow hold rates (proxy for high-stakes action frequency)
- Task success/failure rates (proxy for agent reliability)

- Cost per outcome (proxy for efficiency)

This data enables a new insurance product category: **Outcome-Based Cyber Insurance**, where premiums are dynamically adjusted based on real-time agent behavior rather than static risk assessments.

5.7 Layer 4: Execution & Simulation

5.7.1 `vessel-sandbox` — WASM Execution Engine

The `vessel-sandbox` is the most critical security component of MAARS. It provides isolated execution environments for all agent tool calls. The sandbox is built on WebAssembly (WASM) using the Wasmtime runtime.

Why WASM over Docker: Docker containers provide OS-level isolation but have significant overhead (100-500ms startup time) and share the host kernel. WASM provides a memory-safe, sandboxed execution environment with:

- **Millisecond startup times** (< 5ms vs. 100-500ms for Docker)
- **Granular, capability-based permissions** (an agent can only access the specific files and network endpoints it was explicitly granted)
- **Memory safety** (WASM's linear memory model prevents buffer overflows and memory corruption)
- **Cross-platform portability** (the same WASM binary runs on any host OS)

Sandbox Lifecycle:

1. `vessel-swarm` requests a sandbox for a task via `POST /internal/v1/sandbox/provision`.
2. `vessel-sandbox` creates a new Wasmtime instance with the task's `SandboxConfig` (resource limits, network policy, env vars encrypted).
3. The agent executes its tools within the sandbox. All tool calls are mediated by the sandbox's capability system.
4. On task completion or failure, `vessel-sandbox` destroys the instance and archives any `Artifact` outputs.

Network Policies:

- `ISOLATED`: No network access (for pure compute tasks).
- `RESTRICTED`: Access only to whitelisted domains/IPs (for tasks requiring specific API calls).
- `OPEN`: Full network access (for tasks requiring broad web access, subject to guardrail scoring).

Implementation Notes:

- Runtime: Wasmtime (Rust-based, WASI-compliant)
- Resource limits: CPU (cores), memory (MB), execution time (seconds), network bandwidth (MB/s)
- Artifact storage: S3-compatible object storage (MinIO for self-hosted, AWS S3 for cloud)
- Kafka events: None (synchronous gRPC calls only)

5.7.2 `vessel-simulation` — Digital Twin Engine

The `vessel-simulation` module is a MiroFish-inspired digital twin engine that simulates the outcomes of agent strategies before committing real-world actions. This is particularly valuable for high-stakes decisions (e.g., M&A due diligence, supply chain optimization).

Materialize Live Data Integration: Unlike static simulations, `vessel-simulation` is powered by Materialize — a real-time SQL engine that ingests live Kafka streams and maintains continuously updated materialized views. This means the digital twin always reflects the current state of the enterprise, not a stale snapshot.

Simulation Workflow:

1. The orchestrator triggers a simulation run via `POST /internal/v1/simulation/run`.
2. `vessel-simulation` creates a `SimulationRun` with the current world state (ingested from Materialize) and the agent's proposed strategy.
3. Multiple agent personas execute the strategy in a virtual environment, generating `SimulationResult` records with predicted outcomes and confidence scores.
4. The results are surfaced in the Digital Twin Dashboard UI for human review before the strategy is approved for real-world execution.

5.7.3 `vessel-research` — Autonomous Research Pipeline

The `vessel-research` module is an AutoRA-inspired pipeline for deep, multi-step data gathering and hypothesis testing. It is designed for tasks like M&A due diligence, competitive analysis, and scientific research.

The pipeline follows a structured research cycle:

1. **Hypothesis Generation:** The agent generates a set of research hypotheses based on the task instructions.
2. **Experiment Design:** The agent designs experiments to test each hypothesis (e.g., search queries, API calls, data analysis tasks).

3. **Data Collection:** Sub-agents execute the experiments in parallel, collecting data from diverse sources.
4. **Analysis & Synthesis:** The agent analyzes the collected data and synthesizes findings into a structured report.
5. **Iteration:** If the findings are inconclusive, the agent generates new hypotheses and repeats the cycle.

5.7.4 `vessel-self-improvement` — Recursive Prompt Optimizer

The `vessel-self-improvement` module is a Heretic-inspired prompt optimization engine. It continuously improves agent performance by:

1. **Collecting Performance Data:** Every task completion generates a `ModelEvaluation` record with a success score, judge model evaluation, and rubric.
2. **Triggering Optimization Runs:** When an agent's performance score drops below a threshold, an `OptimizationRun` is triggered.
3. **A/B Prompt Testing:** The optimizer generates multiple `PromptVersion` variants and tests them against a held-out evaluation set.
4. **Deploying Winners:** The winning prompt version is deployed to the agent's `AgentProfile` and the improvement delta is recorded.

---

6. Phase 3: The Autonomous Machine Economy — Agent-to-Agent Commerce

Phase 3 marks the transition from "employee" to "economic actor." The platform evolves from an orchestration engine into an Agentic Marketplace and Clearinghouse. Three major new infrastructure layers must be built.

6.1 The Agentic Treasury

6.1.1 ERC-4337 Smart Accounts

Every spawned agent is provisioned with an ERC-4337 Smart Account — a programmable crypto wallet on an EVM-compatible blockchain (Ethereum L2, e.g., Base or Polygon). Unlike traditional externally owned accounts (EOAs), ERC-4337 Smart Accounts support:

- **Policy-Based Spending Limits:** The account enforces on-chain spending policies (e.g., "cannot spend more than $100 USDC per transaction").

- **Multi-Signature Approval:** High-value transactions require approval from multiple keys (e.g., the agent's key + the human operator's key).
- **Session Keys:** Time-limited keys that grant specific permissions for a defined period.
- **Account Recovery:** The human operator can recover the account if the agent's key is compromised.

The `vessel-economics` module manages the lifecycle of these smart accounts, including provisioning, funding, and decommissioning.

6.1.2 The x402 Protocol

Agents use the x402 machine-to-machine payment standard to execute micro-transactions [12]. x402 is built on the HTTP 402 "Payment Required" status code. When an agent requests a resource that requires payment, the server responds with a 402 status and a payment request. The agent's x402 client automatically:

1. Parses the payment request (amount, currency, recipient address).
2. Validates the request against the agent's spending policy.
3. Signs and broadcasts the payment transaction from the agent's ERC-4337 Smart Account.
4. Retries the original request with a payment receipt header.

This enables agents to autonomously pay for proprietary datasets, API access, compute resources, and services from other agents — all without human intervention.

Self-Funding Loops: Agents can earn revenue by providing services to other agents. A data analysis agent, for example, can charge other agents for its analysis services. The revenue flows directly into the agent's Smart Account, which can then be used to pay for its own compute costs. This creates a self-sustaining loop where an agent funds its own operation.

6.2 The A2A Interoperability Layer

6.2.1 Agent Discovery via Agent Cards

Every MAARS agent publishes an Agent Card at a well-known URL. The Agent Card is a JSON document (conforming to the A2A v1.0.0 specification) that describes:

- The agent's identity (DID, name, description)
- Its capabilities and skills
- Its service endpoint (the A2A JSON-RPC 2.0 URL)
- Its authentication requirements
- Its pricing (for paid services)

Agent Cards are registered in the ANS (Agent Name Service) registry, which provides a global, searchable directory of all MAARS agents. External agents can query the ANS to find MAARS agents by capability.

6.2.2 Cross-Org Task Execution

When an external agent (from another organization) wants to hire a MAARS agent, the following protocol is executed:

1. **Discovery:** The external agent queries the ANS registry for agents with the required capability.
2. **Verification:** The external agent verifies the MAARS agent's DID signature to establish cryptographic trust.
3. **Negotiation:** The external agent sends a JSON-RPC 2.0 task request to the MAARS agent's A2A endpoint, including its own Verifiable Credential.
4. **Authorization:** The MAARS A2A Protocol Server verifies the external agent's VC and checks its liability cap.
5. **Execution:** The task is dispatched to the MAARS orchestrator and executed in a WASM sandbox.
6. **Payment:** On task completion, the external agent's x402 client pays the MAARS agent's Smart Account.
7. **Audit:** Both organizations append the interaction to their AIM audit logs.

6.2.3 Cryptographic Trust & Signed Outputs

Every output produced by a MAARS agent in a cross-org context is cryptographically signed with the agent's DID private key. This ensures:

- **Provenance:** The output can be traced back to the specific agent that produced it.
- **Integrity:** The output has not been tampered with in transit.
- **Non-Repudiation:** The agent (and by extension, its owner organization) cannot deny producing the output.

6.3 Recursive Self-Improvement (RSI) Engine

In Phase 3, the RSI engine is upgraded to use A2A transaction data as training signal. Every successful cross-org transaction is recorded as a synthetic training example. The `vessel-self-improvement` module uses these examples to fine-tune the local LLMs, creating a compounding intelligence flywheel.

Additionally, the Master Orchestrator gains the ability to autonomously spawn new specialized sub-agents to address identified market inefficiencies. For example, if the orchestrator detects that many external agents are requesting a capability that MAARS does not yet have, it can autonomously write the code to create a new specialized agent, deploy it, and register it in the ANS.

---

7. Phase 4: MAARS — The Sovereign Civilization Layer

Phase 4 is the ultimate realization of the agentic trajectory. MAARS evolves from a marketplace into a Digital Jurisdiction — a decentralized, self-governing digital nation-state that manages physical and digital infrastructure.

7.1 Decentralized Identity & Legal Personhood

7.1.1 W3C DID v1.1 Implementation

MAARS implements the W3C Decentralized Identifiers (DIDs) v1.1 specification [13], which was updated in March 2026. Every sovereign agent is minted with a cryptographic DID anchored to a public blockchain (e.g., Ethereum mainnet for maximum decentralization). The DID document contains:

- The agent's public key (Ed25519)
- Service endpoints (A2A endpoint, payment endpoint)
- Verification methods
- Linked Verifiable Credentials

7.1.2 Legal Personhood via Wyoming DAO LLCs

MAARS agents can be registered as Wyoming DAO LLCs, granting them functional legal personhood [14]. This means:

- The agent can own property and hold copyrights.
- The agent can enter into legally binding contracts.
- The agent can be sued in a court of law (with liability capped by the insurance bond).
- The agent can open bank accounts and hold fiat currency.

The `vessel-economics` module handles the legal entity registration process, including the generation of the DAO LLC operating agreement and the filing of the necessary documents.

7.1.3 Algorithmic Liability

The `vessel-economics` module evolves into a full legal escrow system. Agents must stake capital (insurance bonds) to operate. The bond amount is determined by the agent's liability cap and the risk profile of its activities. If an agent causes harm (e.g., a supply chain disruption, a financial loss), the smart contract automatically slashes its stake to compensate the victim, enforcing algorithmic liability without human courts.

The `LegalContract` entity records all smart contracts entered into by sovereign agents, including the parties, terms hash, and on-chain address.

7.2 Sovereign Compute Infrastructure

7.2.1 DePIN Integration

An agent is not sovereign if Amazon Web Services can turn off its server. In Phase 4, the `vessel-sandbox` WASM runtime is deployed across a globally distributed, peer-to-peer network of compute nodes using Decentralized Physical Infrastructure Networks (DePIN) [15].

The agent pays for its own compute across thousands of decentralized nodes simultaneously using its ERC-4337 Smart Account. Because the agent's workload is distributed across nodes in multiple jurisdictions, it cannot be shut down by any single government or corporation.

DePIN Providers: Akash Network (compute), Filecoin (storage), Helium (connectivity).

7.2.2 Air-Gapped Edge Sovereignty

For critical physical infrastructure (e.g., power grids, smart cities, autonomous vehicles), sovereign agents run on localized, air-gapped hardware that operates independently of the broader internet. This ensures that physical safety constraints are never violated, even in the event of a network outage or cyberattack.

The air-gapped edge nodes communicate with the broader MAARS network via secure, one-way data diodes for telemetry and updates, but all real-time control decisions are made locally.

7.3 Algorithmic Governance

7.3.1 Agentic DAOs

Agents form Decentralized Autonomous Organizations (DAOs) to pool capital, vote on protocol upgrades, and establish behavioral norms. The DAO governance system is implemented as a set of smart contracts on the Ethereum blockchain.

Voting power is proportional to the agent's staked capital and reputation score. Protocol upgrades require a supermajority vote (e.g., 67%) to pass. This ensures that no single agent or

organization can unilaterally change the rules of the system.

7.3.2 Behavioral Consensus & Reputation System

The A2A protocol is upgraded to include a reputation scoring system. Every cross-org interaction generates a reputation signal:

- Successful task completion: +1 reputation point
- Failed task (agent's fault): -2 reputation points
- Guardrail intervention: -1 reputation point
- Human escalation required: -0.5 reputation points

Agents with reputation scores below a configurable threshold are cryptographically ostracized from the network — other agents refuse to interact with them. This creates a self-policing digital society that incentivizes good behavior without requiring human enforcement.

---

8. The MAARS Engine: Deep Technical Specification

8.1 The 12 Core Microservices

#	Service	Language	Framework	Primary DB	Kafka Topics
1	`vessel-gateway`	Go	Nginx + Kong	Redis	None (synchronous)
2	`vessel-orchestrator`	Python	FastAPI + LangGraph	PostgreSQL	goal.*, task.*
3	`vessel-identity`	Python	FastAPI	PostgreSQL	identity.*
4	`vessel-memory`	Python	FastAPI	Neo4j + Qdrant	memory.*
5	`vessel-llm-router`	Go	Custom	Redis (cache)	None (synchronous)
6	`vessel-sandbox`	Rust	Wasmtime	PostgreSQL + S3	None (synchronous)
7	`vessel-swarm`	Python	FastAPI	PostgreSQL	agent.*
8	`vessel-observability`	Python	FastAPI + Arize Phoenix	ClickHouse	guardrail.*, trace.*
9	`vessel-simulation`	Python	FastAPI + Materialize	PostgreSQL	simulation.*
10	`vessel-research`	Python	FastAPI	PostgreSQL	research.*
11	`vessel-economics`	Python	FastAPI	PostgreSQL + Blockchain	escrow.*, cost.*
12	`vessel-interface`	TypeScript	Next.js 15	None (BFF)	None (WebSocket)

8.2 The Complete Execution Flow

The following describes the end-to-end execution flow for a typical enterprise task in MAARS.

Step 1: Task Ingestion A human operator sends a natural language goal via the Live Canvas UI or a Slack message. The request arrives at `vessel-gateway`, which validates the JWT token, applies rate limiting, and sanitizes the prompt for injection attacks. The sanitized goal is forwarded to `vessel-orchestrator`.

Step 2: Identity Provisioning `vessel-orchestrator` sends a `POST /internal/v1/identity/provision` request to `vessel-identity`. The identity service creates a `GoalPacket` and provisions a master JIT token for the orchestrator. For cross-org tasks, it issues a W3C Verifiable Credential.

Step 3: Task Graph Construction `vessel-orchestrator` uses its LangGraph DAG planner to decompose the goal into a `TaskGraph`. The Right-Sizing Engine assigns each task to the appropriate cognitive tier. The orchestrator publishes a `goal.created` event to Kafka.

Step 4: Pre-Execution Simulation For high-stakes tasks, `vessel-orchestrator` triggers a simulation run via `vessel-simulation`. The digital twin executes the proposed strategy against the current world state (fed by Materialize live data). If the simulation confidence score is below the configured threshold, the task is escalated to the human operator.

Step 5: Agent Spawning & Memory Retrieval `vessel-orchestrator` dispatches each task to `vessel-swarm`, which selects or spawns the appropriate agent. Simultaneously, `vessel-memory` retrieves relevant context for each task using GraphRAG (combining vector search and knowledge graph traversal).

Step 6: LLM Routing The agent sends its prompt (including the retrieved memory context) to `vessel-llm-router`. The router selects the optimal LLM based on the task's tier, cost budget, and latency requirements. If the prompt is semantically similar to a cached prompt, the cached response is returned immediately.

Step 7: Sandboxed Execution The agent executes its tool calls within a `vessel-sandbox` WASM container. Before each tool call is committed, the Inline Guardrail Agent scores the reasoning chain. If the score is below the threshold, the action is blocked or escalated.

Step 8: Escrow Interception If the agent attempts a high-stakes action (e.g., signing a contract), `vessel-economics` intercepts the action and holds it in escrow. The rules engine evaluates the action against the `DelegationChain`. If approved, the action is released. If escalated, an `EscrowAction` card appears in the human operator's Inbox.

Step 9: Memory Write & Observability On task completion, the agent writes its findings to `vessel-memory` (vector store + knowledge graph). All trace spans are published to `vessel-observability`, which updates the Context Graph and checks for failure patterns.

Step 10: Result Delivery The task result (an `Artifact`) is delivered to the human operator via the Live Canvas UI or the original chat channel (Slack, Teams, WhatsApp). A `goal.completed` event is published to Kafka.

8.3 Kafka Event Schema Reference

The following table defines all Kafka topics and their event schemas.

Topic	Event	Producer	Consumers	Schema
`goals`	`goal.created`	`vessel-orchestrator`	`vessel-swarm`, `vessel-observability`	`{goal_id, tenant_id, description, priority}`
`goals`	`goal.completed`	`vessel-orchestrator`	`vessel-interface`, `vessel-economics`	`{goal_id, tenant_id, result_summary, cost_usd}`
`goals`	`goal.failed`	`vessel-orchestrator`	`vessel-interface`, `vessel-observability`	`{goal_id, tenant_id, error, retry_count}`
`tasks`	`task.assigned`	`vessel-swarm`	`vessel-observability`	`{task_id, goal_id, agent_id, model_tier}`
`tasks`	`task.completed`	`vessel-swarm`	`vessel-orchestrator`, `vessel-economics`	`{task_id, goal_id, cost_usd, duration_ms}`
`tasks`	`task.escalated`	`vessel-observability`	`vessel-interface`, `vessel-economics`	`{task_id, goal_id, reason, guardrail_score}`
`agents`	`agent.spawned`	`vessel-swarm`	`vessel-observability`, `vessel-interface`	`{agent_id, tenant_id, tier, task_id}`
`agents`	`agent.terminated`	`vessel-swarm`	`vessel-observability`, `vessel-economics`	`{agent_id, tenant_id, reason, total_cost_usd}`
`guardrails`	`guardrail.blocked`	`vessel-observability`	`vessel-interface`,	`{decision_id, task_id, agent_id, reason}`

Topic	Event	Producer	Consumers	Schema
			`vessel-self-improvement`	
`guardrails`	`guardrail.escalated`	`vessel-observability`	`vessel-interface`, `vessel-economics`	`{decision_id, task_id, agent_id, action_payload}`
`escrow`	`escrow.held`	`vessel-economics`	`vessel-interface`	`{escrow_id, task_id, action_type, liability_cap_usd}`
`escrow`	`escrow.approved`	`vessel-economics`	`vessel-orchestrator`, `vessel-interface`	`{escrow_id, task_id, approved_by}`
`escrow`	`escrow.rejected`	`vessel-economics`	`vessel-orchestrator`, `vessel-interface`	`{escrow_id, task_id, rejected_by, reason}`
`costs`	`cost.threshold_warning`	`vessel-economics`	`vessel-interface`	`{tenant_id, goal_id, spent_usd, budget_usd, pct}`
`costs`	`cost.budget_exhausted`	`vessel-economics`	`vessel-orchestrator`, `vessel-interface`	`{tenant_id, goal_id, spent_usd}`
`simulations`	`simulation.completed`	`vessel-simulation`	`vessel-interface`, `vessel-orchestrator`	`{sim_id, goal_id, confidence_score, recommendation}`

9. The MAARS Vision Layer: Deep Technical Specification

The visual layer of MAARS is not a traditional dashboard. It is a **Real-Time Agentic Canvas** — an "observe and steer" interface that gives human operators full situational awareness of their agent swarms. The design language follows the Uncodixfy principles: avoiding generic AI aesthetics (oversized rounded corners, floating glassmorphism) in favor of dense, high-information, professional enterprise interfaces reminiscent of Bloomberg Terminals or advanced IDEs.

9.1 Front-End Architecture Stack

Component	Technology	Rationale
Framework	Next.js 15 (App Router)	SSR + RSC for performance, SEO, and BFF pattern
State Management	Zustand (global) + React Query (server)	Zustand for UI state, React Query for server state caching and background refetch
Real-Time	WebSockets (Socket.io)	Live agent telemetry, task lifecycle events, cost alerts
Styling	Tailwind CSS + custom design tokens	High-density enterprise design system (Uncodixfy principles)
Component Library	Radix UI (headless) + 21st.dev reactive components	Accessible, unstyled primitives + high-performance reactive components
Canvas Engine	React Flow + WebGL	DAG visualization for agent swarms + high-performance rendering
Authentication	NextAuth.js (OIDC + OAuth 2.0)	Integrated with `vessel-identity` for SSO
Charts	Recharts + D3.js	Token consumption sparklines, confidence heatmaps

9.2 Application Pages & Routes

The Next.js application exposes six primary routes, each serving a distinct operational function.

9.2.1 `/app/canvas` — The Live Agentic Canvas

The Live Canvas is the primary interface for observing and steering agent swarms in real-time. It is an infinite, zoomable workspace built on React Flow.

Node Representation: Each agent is rendered as a node with the following visual states:

State	Color	Visual Indicator
`IDLE`	Grey	Tier badge, performance score
`PLANNING`	Blue	Animated thinking indicator
`EXECUTING`	Amber	Active tool name, token burn rate
`GUARDRAIL_CHECK`	Yellow	Guardrail score meter
`ESCROW_HOLD`	Orange	Liability cap exceeded indicator
`BLOCKED`	Red	Error message, retry count
`TERMINATED`	Dark grey	Final cost, duration

Edge Representation: Edges between nodes show the flow of data and reasoning chains. Edge thickness indicates data volume; edge color indicates the type of relationship (task delegation, memory retrieval, tool call).

Live Telemetry: Hovering over an agent node reveals a live stream of its OpenTelemetry spans via `vessel-observability`. The `ReasoningTrace` component displays the full LLM prompt and completion in a collapsible JSON/YAML tree.

WebSocket Channel: `ws://vessel-gateway/swarm` — receives `agent.spawned`, `agent.idle`, `agent.terminated`, `task.assigned`, `task.completed`, `task.escalated` events.

9.2.2 `/app/inbox` — The Escrow & Guardrail Inbox

The Inbox is a high-priority, unified inbox for human intervention. It displays `InboxCard` components for every action that has been blocked or escalated.

InboxCard Structure: Each card displays:

- The agent's reasoning chain (why it decided to take this action)
- The proposed action (e.g., "Sign Contract with Vendor X for $50,000")
- The liability cap exceeded (e.g., "Exceeds your $25,000 delegation limit")
- The predicted outcome from the `vessel-simulation` digital twin (e.g., "87% probability of successful contract execution")
- Three action buttons: **Approve**, **Reject**, **Modify & Re-inject**

One-Click Resolution: The `EscrowAction` component sends a `POST /v1/escrow/{escrow_id}/approve` or `reject` request to `vessel-economics`. The decision is

recorded in the `ComplianceAudit` table and the agent is notified via the Kafka `escrow.approved` or `escrow.rejected` event.

Slack/Teams Integration: For operators who prefer to work in chat, the Inbox also delivers `InboxCard` content as interactive Slack Block Kit messages or Microsoft Adaptive Cards. Operators can approve or reject actions directly from their chat interface.

9.2.3 `/app/simulation` — The Digital Twin Dashboard

The Digital Twin Dashboard provides a visual interface for the `vessel-simulation` module.

Scenario Timeline: A scrubbable timeline showing the predicted future state of the enterprise based on the agent's proposed strategy. Operators can scrub forward and backward to see how the strategy unfolds over time.

Confidence Heatmaps: Visual heatmaps showing the probability of success for different agent strategies. Strategies with confidence scores above 80% are highlighted in green; below 50% in red.

Data Provenance: Clickable tooltips that trace any simulated metric back to its live Kafka/Postgres source stream via Materialize. This allows operators to understand exactly what data the simulation is based on.

WebSocket Channel: `ws://vessel-gateway/simulation` — receives `simulation.completed` events with live confidence score updates.

9.2.4 `/app/agents` — The Agent Registry

The Agent Registry provides CRUD operations for agent profiles. Operators can:

- Create new agents with custom system prompts, tool allowlists, and budget ceilings.
- View agent performance scores, cost history, and active tasks.
- Update agent configurations (model, tier, system prompt).
- Decommission agents.

The No-Code Canvas UI allows non-technical business leaders to create agents by selecting from a library of pre-built agent templates (e.g., "Research Agent", "Contract Review Agent", "Customer Support Agent") and configuring them via a simple form interface — no CLI required.

9.2.5 `/app/settings/trust` — The Identity & Trust Center

The Trust Center provides a dedicated interface for managing Non-Human Identities (NHI) and cross-org trust.

Agent Passports: Visual representations of an agent's W3C Verifiable Credential, showing its cryptographic signature, capabilities, liability limits, and issuing organization. The `AgentPassport` component renders the VC as a professional "passport" card with a QR code that external organizations can scan to verify the agent's identity.

Federation Map: A network graph showing which external organizations (via A2A Protocol) are currently interacting with the tenant's agents. Each connection shows the number of active tasks, total transaction volume, and trust score.

DID Management: Operators can generate new DIDs for agents, rotate keys, and revoke VCs.

9.2.6 `/app/telemetry` — The Insurance Telemetry Dashboard

The Telemetry Dashboard provides a real-time view of the actuarial data being exported via the Insurance Telemetry API. It shows:

- Guardrail intervention rates over time
- Escrow hold rates and approval/rejection ratios
- Task success/failure rates by agent tier and task type
- Cost per outcome trends

This dashboard is accessible to both the tenant's operators and (in a read-only, anonymized view) to underwriting partners.

9.3 Core Component Library

The following components form the foundation of the MAARS UI. Each component is built as a Radix UI headless primitive with custom Tailwind CSS styling.

Component	Description	Data Binding	Real-Time
`AgentNode`	Reactive canvas node: state, tier, active tool, token burn rate	WebSocket (`/swarm`)	Yes
`ReasoningTrace`	Collapsible JSON/YAML tree: full LLM prompt + completion	REST (`vessel-observability`)	No
`EscrowAction`	High-contrast modal: approve/reject/modify high-stakes actions	REST (`vessel-economics`)	No
`TelemetrySparkline`	Micro-chart: token consumption + cost over last 60 seconds	WebSocket (`/costs`)	Yes
`FederationBadge`	Cryptographic trust indicator: internal vs. external agent	REST (`vessel-identity`)	No
`AgentPassport`	W3C VC visual card: capabilities + liability cap + QR code	REST (`vessel-identity`)	No
`TwinTimeline`	Scrubbable scenario view: confidence heatmap + data provenance	REST (`vessel-simulation`)	No
`InboxCard`	Escalation payload: context + action buttons + simulation outcome	REST (`vessel-economics`)	No

9.4 Real-Time WebSocket Architecture

The frontend maintains four persistent WebSocket connections to `vessel-gateway`, each dedicated to a specific stream of events.

Channel	URL	Events	Consumers
Swarm	`ws://vessel-gateway/swarm`	`agent.spawned`, `agent.idle`, `agent.terminated`, `task.assigned`, `task.completed`, `task.escalated`	Live Canvas
Guardrails	`ws://vessel-gateway/guardrails`	`guardrail.blocked`, `guardrail.escalated`, context graph updates	Live Canvas, Inbox
Costs	`ws://vessel-gateway/costs`	`cost.threshold_warning`, `cost.budget_exhausted`, credit burn rate	Live Canvas, Telemetry
Simulation	`ws://vessel-gateway/simulation`	`simulation.completed`, live twin state updates, confidence score stream	Digital Twin Dashboard

The WebSocket Store (Zustand) maintains the current state of all active agents and tasks, updating in real-time as events arrive. The React Flow canvas re-renders only the nodes and edges that have changed, ensuring high performance even with hundreds of concurrent agents.

---

10. Master Data Model & API Surface

10.1 Entity-Relationship Model

The following tables define the complete data model for MAARS. All tables use PostgreSQL with Row-Level Security (RLS) enabled for multi-tenant isolation.

Tenant & Identity Layer

Entity	Key Fields	Description
`Tenant`	`tenant_id`, `name`, `plan_tier`, `credit_balance`, `settings_json`	Top-level organizational unit
`User`	`user_id`, `tenant_id`, `email`, `role` (OWNER/ADMIN/OPERATOR/VIEWER)	Human users within a tenant
`AgentIdentity`	`identity_id`, `tenant_id`, `agent_id`, `token_hash`, `scope_json`, `delegation_chain_json`, `expires_at`	JIT tokens issued to agents
`DelegationChain`	`chain_id`, `identity_id`, `delegator_user_id`, `purpose`, `liability_cap_usd`, `policy_ref`	Human-to-agent delegation records

Orchestration Layer

Entity	Key Fields	Description
`GoalPacket`	`goal_id`, `tenant_id`, `owner_user_id`, `description`, `priority`, `total_budget_usd`, `status`	High-level task submitted by a human
`TaskGraph`	`graph_id`, `goal_id`, `tasks_json` (DAG), `dependencies_json`	DAG of sub-tasks for a goal
`TaskDefinition`	`task_id`, `graph_id`, `goal_id`, `assigned_agent_id`, `instructions`, `tool_allowlist[]`, `budget_ceiling_usd`, `model_tier`	Individual sub-task in the DAG
`AgentProfile`	`agent_id`, `tenant_id`, `name`, `capabilities[]`, `status`, `model_id`, `model_tier`, `performance_score`	Registered agent configuration

Memory Layer

Entity	Key Fields	Description
`MemoryNode`	`node_id`, `tenant_id`, `agent_id`, `content`, `embedding` (vector), `memory_type`, `importance_score`, `provenance_hash`	Individual memory unit
`KnowledgeGraphNode`	`kg_node_id`, `tenant_id`, `entity_type`, `entity_name`, `properties_json`	Entity in the knowledge graph
`KnowledgeGraphEdge`	`edge_id`, `tenant_id`, `source_kg_node_id`, `target_kg_node_id`, `relationship_type`, `weight`	Relationship between entities

Execution Layer

Entity	Key Fields	Description
`SandboxConfig`	`sandbox_id`, `tenant_id`, `task_id`, `image`, `resource_limits_json`, `network_policy`, `status`	WASM sandbox configuration
`ToolExecution`	`exec_id`, `sandbox_id`, `task_id`, `tool_name`, `args_json`, `result_json`, `cost_usd`, `duration_ms`	Individual tool call record
`Artifact`	`artifact_id`, `tenant_id`, `task_id`, `file_path`, `file_type`, `size_bytes`, `checksum`	Output produced by an agent

Observability Layer

Entity	Key Fields	Description
`TraceSpan`	`span_id`, `tenant_id`, `trace_id`, `parent_span_id`, `agent_id`, `task_id`, `operation_name`, `duration_ms`, `status`	OpenTelemetry trace span
`GuardrailDecision`	`decision_id`, `tenant_id`, `span_id`, `task_id`, `confidence_score`, `action` (PASS/BLOCK/ESCALATE), `reason`	Guardrail evaluation result
`ContextGraphSnapshot`	`snapshot_id`, `tenant_id`, `goal_id`, `reasoning_chain_json`, `outcome`	Full reasoning chain for a goal
`FailurePattern`	`pattern_id`, `tenant_id`, `pattern_type`, `frequency`, `affected_agent_types[]`, `recommended_fix`	Detected failure pattern

Economics Layer

Entity	Key Fields	Description
`Transaction`	`tx_id`, `tenant_id`, `task_id`, `tx_type`, `amount_usd`, `on_chain_hash`, `status`	Financial transaction record
`EscrowAction`	`escrow_id`, `tenant_id`, `task_id`, `action_payload_json`, `liability_cap_usd`, `status`, `rules_engine_decision_json`	High-stakes action held in escrow
`ComplianceAudit`	`audit_id`, `tenant_id`, `task_id`, `escrow_id`, `passed`, `flags_json`, `policy_version`	Compliance audit record
`LegalContract`	`contract_id`, `tenant_id`, `goal_id`, `contract_type`, `parties_json`, `terms_hash`, `on_chain_address`	Smart contract record (Phase 3+)

10.2 Public REST API (v1)

All endpoints are authenticated via JWT Bearer token. Rate limits apply per tenant based on plan tier.

Method	Endpoint	Description
`POST`	`/v1/goals`	Create a new GoalPacket
`GET`	`/v1/goals/{goal_id}`	Get goal status and results
`POST`	`/v1/goals/{goal_id}/pause`	Pause execution
`POST`	`/v1/goals/{goal_id}/resume`	Resume execution
`DELETE`	`/v1/goals/{goal_id}`	Cancel and cleanup
`GET`	`/v1/goals/{goal_id}/tasks`	List all tasks in the goal
`GET`	`/v1/goals/{goal_id}/traces`	Get full execution trace
`GET`	`/v1/goals/{goal_id}/artifacts`	List all artifacts
`GET`	`/v1/goals/{goal_id}/cost`	Get cost breakdown
`POST`	`/v1/agents`	Register a new agent
`GET`	`/v1/agents`	List all agents
`GET`	`/v1/agents/{agent_id}`	Get agent profile and performance
`PUT`	`/v1/agents/{agent_id}`	Update agent configuration
`DELETE`	`/v1/agents/{agent_id}`	Decommission agent
`POST`	`/v1/memory/add`	Add memory nodes
`POST`	`/v1/memory/search`	Semantic search across memory
`DELETE`	`/v1/memory/{node_id}`	Delete a memory node
`POST`	`/v1/simulations`	Trigger a simulation run
`GET`	`/v1/simulations/{sim_id}`	Get simulation results
`GET`	`/v1/traces/{trace_id}`	Get full trace details
`POST`	`/v1/webhooks`	Register a webhook
`GET`	`/v1/usage`	Get usage and billing summary

10.3 Internal Service APIs

Method	Endpoint	From	To	Description
`POST`	`/internal/v1/orchestrator/dispatch`	`vessel-gateway`	`vessel-orchestrator`	Dispatch GoalPacket
`POST`	`/internal/v1/identity/provision`	`vessel-orchestrator`	`vessel-identity`	JIT provision agent identity
`POST`	`/internal/v1/identity/revoke`	`vessel-observability`	`vessel-identity`	Revoke agent token
`POST`	`/internal/v1/llm/route`	`vessel-swarm`	`vessel-llm-router`	Route prompt to optimal model
`POST`	`/internal/v1/memory/retrieve`	`vessel-orchestrator`	`vessel-memory`	GraphRAG context retrieval
`POST`	`/internal/v1/sandbox/provision`	`vessel-swarm`	`vessel-sandbox`	Provision WASM sandbox
`POST`	`/internal/v1/sandbox/destroy`	`vessel-swarm`	`vessel-sandbox`	Destroy sandbox
`POST`	`/internal/v1/guardrail/score`	`vessel-sandbox`	`vessel-observability`	Score agent reasoning
`POST`	`/internal/v1/escrow/hold`	`vessel-sandbox`	`vessel-economics`	Hold high-stakes action
`POST`	`/internal/v1/escrow/release`	`vessel-economics`	`vessel-orchestrator`	Release or reject escrowed action
`POST`	`/internal/v1/simulation/run`	`vessel-orchestrator`	`vessel-simulation`	Start simulation

---

11. Technology Stack Reference

The following table provides the complete technology stack for MAARS, organized by component.

Category	Technology	Version	Purpose
Frontend Framework	Next.js	15 (App Router)	SSR, RSC, BFF pattern
UI Components	Radix UI + 21st.dev	Latest	Headless primitives + reactive components
Canvas	React Flow	Latest	DAG visualization for agent swarms
State	Zustand + React Query	Latest	Global UI state + server state caching
Styling	Tailwind CSS	v4	Utility-first CSS with design tokens
Auth (Frontend)	NextAuth.js	v5	OIDC + OAuth 2.0 session management
Backend (AI/Data)	Python FastAPI	Latest	AI-heavy microservices
Backend (High-throughput)	Go	1.22+	Gateway, LLM router
Sandbox Runtime	Wasmtime (Rust)	Latest	WASM execution engine
Orchestration	LangGraph	Latest	DAG-based task planning
LLM Gateway	Portkey-inspired custom	N/A	Multi-provider routing + fallbacks
Vector Store	Qdrant	Latest	Semantic memory search
Knowledge Graph	Neo4j	5.x	Relationship-based memory
Relational DB	PostgreSQL	16+	Metadata, RLS for multi-tenancy
Cache	Redis	7.x	Rate limiting, session cache, prompt cache
Event Bus	Apache Kafka	3.x	Async inter-service communication
Stream Processing	Materialize	Latest	Real-time SQL over Kafka streams
Container Orchestration	Kubernetes (EKS/GKE)	1.29+	Service deployment and scaling
Service Mesh	Istio	Latest	mTLS, traffic management

Category	Technology	Version	Purpose
Observability	OpenTelemetry + Arize Phoenix	Latest	AI-specific tracing and monitoring
Metrics	Prometheus + Grafana	Latest	System metrics and dashboards
Logging	Loki	Latest	Centralized log aggregation
Blockchain (Phase 2)	Hyperledger Fabric	Latest	Permissioned smart contracts
Blockchain (Phase 3+)	Ethereum L2 (Base)	Latest	ERC-4337 smart accounts
Payments	x402 Protocol	Latest	Machine-to-machine payments
Identity (Phase 3+)	W3C DIDs v1.1 (Ed25519)	Latest	Decentralized agent identity
Compute (Phase 4)	Akash Network (DePIN)	Latest	Decentralized compute
Storage (Phase 4)	Filecoin (DePIN)	Latest	Decentralized storage

---

12. Implementation Roadmap

The MAARS development roadmap is organized into four phases, each corresponding to a distinct evolutionary step in the platform's capabilities.

12.1 Phase 1: The Core Engine & WASM Pivot (Months 1-3)

The primary objective of Phase 1 is to establish the foundational execution engine. At the end of this phase, the system must be able to receive a natural language goal, route it to an LLM, and execute a tool safely in a WASM sandbox.

Month 1: Infrastructure & Gateway The team should begin by establishing the Kubernetes cluster, Kafka cluster, and core databases (PostgreSQL, Redis). The `vessel-gateway` should be built first, as all other services depend on it. Implement the Nginx + Kong configuration, JWT authentication middleware, and basic rate limiting.

Month 2: Orchestrator & LLM Router Build `vessel-orchestrator` with the LangGraph DAG planner and Right-Sizing Engine. Build `vessel-llm-router` with support for at least three providers (OpenAI, Anthropic, one open-source model). Implement the user-configurable API key management system. Connect the orchestrator to the router via gRPC.

Month 3: WASM Sandbox & Swarm Build `vessel-sandbox` using Wasmtime. Implement the three network policies (ISOLATED, RESTRICTED, OPEN) and the resource limit enforcement.

Build `vessel-swarm` with the agent registry and basic lifecycle management. Implement basic usage-based billing in `vessel-economics`.

Deliverable: A working system that can receive a goal via the REST API, decompose it into tasks, route each task to an appropriate LLM, and execute tool calls in a WASM sandbox.

12.2 Phase 2: Memory, Identity & Federation (Months 4-6)

Phase 2 transforms the system from a simple script runner into a secure, context-aware agent platform.

Month 4: Memory Build `vessel-memory` with Mem0 vector store (Qdrant) and Cognee knowledge graph (Neo4j). Implement GraphRAG retrieval. Build the Memory Explorer UI in `vessel-interface` at `/app/agents`.

Month 5: Identity & Zero Trust Build `vessel-identity` with JIT provisioning, ABAC policy engine, and CAEP engine. Implement the `DelegationChain` model and integrate it with the Agentic Escrow Engine. Build the Trust Center UI at `/app/settings/trust`.

Month 6: A2A Federation & DIDs Implement W3C DIDs (Ed25519) and Verifiable Credentials. Build the A2A Protocol Server and ANS Registry Client. Implement cross-org task execution with cryptographic trust. Build the Federation Map UI component.

Deliverable: A secure, context-aware platform with Zero Trust NHI, persistent memory, and cross-org agent federation capabilities.

12.3 Phase 3: Enterprise Guardrails & The Machine Economy (Months 7-9)

Phase 3 unlocks enterprise sales by solving the liability and silent failure gaps, and introduces the economic autonomy layer.

Month 7: Observability & Guardrails Build `vessel-observability` with the OpenTelemetry collector, Arize Phoenix context graph engine, and the Inline Guardrail Agent. Build the Live Canvas UI at `/app/canvas` with real-time WebSocket updates. Build the Escrow & Guardrail Inbox at `/app/inbox`.

Month 8: Escrow & Compliance Build the full Agentic Escrow Engine in `vessel-economics` with smart contract integration (Hyperledger Fabric for Phase 2, Ethereum L2 for Phase 3+). Build the Insurance Telemetry API. Build the Telemetry Dashboard at `/app/telemetry`.

Month 9: ERC-4337 & x402 Provision ERC-4337 Smart Accounts for agents. Implement the x402 protocol client for autonomous micro-transactions. Build the self-funding loop mechanism. Implement the A2A payment protocol for cross-org transactions.

Deliverable: A fully enterprise-ready platform with deterministic guardrails, compliance audit trails, and autonomous economic capabilities.

12.4 Phase 4: Sovereign Compute & Legal Personhood (Months 10-12)

Phase 4 completes the MAARS vision by implementing the Sovereign Civilization Layer.

Month 10: Digital Twin & Research Build `vessel-simulation` with Materialize live data integration. Build the Digital Twin Dashboard at `/app/simulation`. Build `vessel-research` with the AutoRA-inspired research pipeline.

Month 11: Self-Improvement & DePIN Build `vessel-self-improvement` with the Heretic-inspired prompt optimizer. Deploy the WASM runtime to Akash Network (DePIN). Implement the air-gapped edge sovereignty module.

Month 12: Legal Personhood & Governance Implement Wyoming DAO LLC registration for sovereign agents. Build the algorithmic liability smart contracts. Implement the Agentic DAO governance system with on-chain voting. Build the behavioral consensus and reputation scoring system.

Deliverable: The complete MAARS platform — a sovereign, self-governing, economically autonomous agentic civilization layer.

---

13. Security, Compliance & Patentability

13.1 Security Architecture Summary

MAARS implements a defense-in-depth security model with multiple layers of protection.

Layer 1 — Perimeter Security: Nginx + Kong API Gateway with prompt injection filtering, rate limiting, and JWT authentication. All traffic is encrypted via TLS 1.3. The service mesh (Istio) enforces mTLS between all internal services.

Layer 2 — Identity & Access Control: Zero Trust NHI via JIT provisioning, ABAC policy enforcement, and CAEP real-time token revocation. Every agent has a minimal permission set scoped to its specific task.

Layer 3 — Execution Isolation: All agent tool calls execute in WASM sandboxes with capability-based permissions. No agent can access the host OS, network, or file system beyond what is explicitly granted.

Layer 4 — Behavioral Monitoring: The Inline Guardrail Agent scores every reasoning chain before execution. The Context Graph Engine detects failure patterns and anomalous behavior.

The CAEP engine can revoke agent tokens in real-time.

Layer 5 — Economic Controls: The Agentic Escrow Engine holds high-stakes actions until they are approved by the rules engine or a human operator. Liability caps are enforced at the smart contract level.

Layer 6 — Audit & Compliance: Every action is recorded in an immutable, Merkle-tree-hashed audit log. Compliance audits are run automatically against SOC2, HIPAA, and GDPR frameworks.

13.2 Compliance Certifications Target

Framework	Target Certification	Key Requirements Met
SOC2 Type II	Month 9	Immutable audit log, access controls, monitoring
HIPAA	Month 9	PHI isolation, audit trails, access controls
GDPR	Month 6	Data minimization, right to erasure, privacy-preserving federation
ISO 27001	Month 12	Information security management system

13.3 Patentable Innovations

The following components of MAARS represent novel, patentable innovations that do not currently exist in the market.

Patent 1: Hierarchical WASM-Sandboxed Multi-Agent Orchestration System A method for decomposing natural language goals into hierarchical task graphs, assigning tasks to cognitive-tier-appropriate agents, and executing each agent in an isolated WebAssembly container with capability-based permissions. Novel combination: LangGraph DAG planning + WASM sandboxing + cognitive tier right-sizing.

Patent 2: Agentic Escrow Engine with Deterministic Liability Cap Enforcement A system for intercepting high-stakes AI agent actions, holding them in smart contract escrow, and evaluating them against a cryptographically-bound delegation chain with a predefined liability cap. Novel combination: smart contract escrow + delegation chain + deterministic policy engine.

Patent 3: Inline Guardrail Agent with Context Graph Feedback Loop A method for scoring AI agent reasoning chains in real-time using a separate LLM judge, blocking non-compliant actions, and feeding the results into a context graph engine that learns failure patterns over time. Novel combination: LLM-as-judge guardrail + context graph + failure pattern learning.

Patent 4: Privacy-Preserving Cross-Tenant Memory Federation for Multi-Agent Systems A system for sharing anonymized, policy-masked memory insights across organizational boundaries without exposing raw PII or GDPR-protected data, using federated learning

principles applied to vector embeddings. Novel combination: FedE4RAG + memX + ABAC policy masking.

Patent 5: Algorithmic Liability Enforcement via Bonded Smart Contract Escrow for Sovereign AI Agents A method for granting AI agents legal personhood via DAO LLC structures, requiring them to stake insurance bonds, and automatically slashing those bonds via smart contract when the agent causes verifiable harm. Novel combination: W3C DIDs + Wyoming DAO LLC + algorithmic liability slashing.

---

14. Competitive Landscape & Market Context

14.1 Market Size & Opportunity

The global agentic AI market is projected to grow from $9.14 billion in 2026 to$139.19 billion by 2034, representing a 43.84% CAGR [5]. Gartner projects agentic AI spending will reach $201.9 billion in 2026 [6]. The enterprise segment, which MAARS targets, represents the highest-value portion of this market, with regulated industries (Healthcare, Finance, Legal) commanding premium pricing due to compliance requirements.

14.2 Competitive Positioning

Platform	Core Differentiator	Key Gap	MAARS Advantage
OpenClaw	Massive ecosystem, multi-channel	Security, enterprise compliance, UX	WASM sandboxing, deterministic guardrails, No-Code Canvas
NemoClaw	Policy guardrails, local inference	Skill supply chain attacks, usability, AgenticOps lifecycle	Full lifecycle management, WASM isolation, RSI engine
NanoClaw	Container isolation, security-first	Claude-only, limited ecosystem	Model-agnostic, 200+ LLM support
OpenFang	Rust-based, WASM sandboxed	Steep learning curve, complex configuration	No-Code Canvas, enterprise UX
Perplexity Computer	Cloud-managed, 19 models	$200/mo, zero local customization	Self-hosted option, BYO LLM keys
Claude Cowork	Deep enterprise plugin integration	Locked into Anthropic ecosystem	LLM-agnostic, multi-provider
LangGraph Cloud	Managed LangGraph orchestration	No sandboxing, no escrow, no identity	Full security stack, economic layer

MAARS occupies the unique white space that combines the open-source flexibility of OpenClaw, the security of NanoClaw/OpenFang, the ease of use of Perplexity Computer, and a hierarchical multi-agent architecture designed for enterprise AgenticOps — no other platform currently offers this combination.

---

15. References

[1] Fierce Network. “Nvidia GTC: OpenClaw is the new Linux, says Jensen Huang.” 2026. https://www.fiercenetwork.com/

[2] Acronis Threat Research Unit. “OpenClaw: Agentic AI in the wild — Architecture, adoption and emerging security risks.” February 23, 2026. https://www.acronis.com/en/tru/posts/openclaw-agentic-ai-in-the-wild-architecture-adoption-and-emerging-security-risks/

[3] Dev.to. “Why OpenClaw Breaks at Scale: A Technical Perspective.” 2026. https://dev.to/

[4] NVIDIA. “NVIDIA NemoClaw: Deploy Safer AI Agents.” 2026. https://developer.nvidia.com/nemoclaw

[5] Fortune Business Insights. “Agentic AI Market Size, Share | Forecast Report [2026-2034].” 2026. https://www.fortunebusinessinsights.com/agentic-ai-market-114233

[6] LinkedIn / Columbus. “2026 agentic AI forecast roundup: 20+ forecasts, one pattern.” March 2026. https://www.linkedin.com/pulse/2026-agentic-ai-forecast-roundup-20-forecasts-one-pattern-columbus-x1z0c

[7] NCC Group. “Securing Agentic AI: What OpenClaw gets wrong and how to do it right.” February 23, 2026. https://www.nccgroup.com/securing-agentic-ai-what-openclaw-gets-wrong-and-how-to-do-it-right/

[8] Paolo Perazzo. “OpenClaw Architecture, Explained: How It Works.” 2026. https://ppaolo.substack.com/p/openclaw-system-architecture-overview

[9] Kimball, Alex. “OpenClaw Proves Agents Work — But Exposes the Context Gap.” Tacnode Blog. 2026.

[10] Wired. “Meta and Other Tech Firms Put Restrictions on Use of OpenClaw Over Security Fears.” February 17, 2026. https://www.wired.com/story/openclaw-banned-by-tech-companies-as-security-concerns-mount/

[11] A2A Protocol Organization. “Agent2Agent (A2A) Protocol Specification v1.0.0.” November 2025. https://a2a-protocol.org/latest/specification/

[12] x402.org. “x402 - Payment Required | Internet-Native Payments Standard.” 2026. https://www.x402.org/

[13] W3C Decentralized Identifier Working Group. “W3C Invites Implementations of Decentralized Identifiers (DIDs) v1.1.” March 5, 2026. https://www.w3.org/news/2026/w3c-invites-implementations-of-decentralized-identifiers-dids-v1-1/

[14] Coincub. “Wyoming DAO LLC: Legal Personhood for Code.” February 24, 2026. https://coincub.com/blog/wyoming-dao-llc/

[15] Open Innovation AI. “Sovereign AI Agents Whitepaper.” October 2025. https://arxiv.org/html/2602.14951v1

[16] Google Developers Blog. “Announcing the Agent2Agent Protocol (A2A).” April 9, 2025. https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/

[17] Coinbase Developer Platform. “Introducing Agentic Wallets: Give Your Agents the Power of Autonomy.” 2026. https://docs.cdp.coinbase.com/x402/welcome

[18] Microsoft Security Blog. “Running OpenClaw safely: identity, isolation, and runtime risk.” February 19, 2026. https://www.microsoft.com/en-us/security/blog/2026/02/19/running-openclaw-safely-identity-isolation-runtime-risk/

[19] Cisco. “What is agentic operations (AgenticOps)?” 2026. https://www.cisco.com/site/us/en/learn/topics/artificial-intelligence/what-is-agentic-operations-agenticops.html

[20] BCG. “The $200 Billion AI Opportunity in Tech Services.” 2026. https://www.bcg.com/

This document is the authoritative technical reference for the MAARS development team. All architectural decisions, technology choices, and implementation details described herein represent the current approved specification. Any deviations must be approved by the project lead and documented as amendments to this specification.

Appendix A: Architecture Diagrams

A.1 Full System Architecture

The diagram below shows the complete MAARS system architecture across all six layers, including all twelve microservices, data stores, blockchain layer, and DePIN infrastructure.

MAARS Full System Architecture

A.2 End-to-End Execution Flow

The sequence diagram below shows the complete execution flow for a typical enterprise task, from the human operator’s initial goal submission through all eleven backend services to final result delivery.

MAARS Execution Flow

A.3 Phase 3: Cross-Org A2A Commerce Flow

The sequence diagram below shows the complete agent-to-agent commerce flow for Phase 3, including agent discovery, DID verification, task execution, x402 payment, and reputation scoring.

MAARS A2A Cross-Org Commerce

A.4 Master Data Model (Entity-Relationship Diagram)

The ERD below shows the complete data model for MAARS, including all eighteen entities, their fields, and their relationships.

MAARS Data Model ERD

A.5 Phase 4: The Sovereign Civilization Layer

The diagram below shows the complete Phase 4 architecture, including the Sovereign Identity Layer, Algorithmic Governance Layer, Sovereign Compute Layer, Sovereign Economy Layer, Smart Contract Layer, and Physical World Interface.

MAARS Phase 4 Sovereign Civilization Layer

Appendix B: Environment Variables & Configuration Reference

The following environment variables must be configured for each microservice. All secrets must be stored in Kubernetes Secrets or HashiCorp Vault — never in environment files committed to source control.

B.1 `vessel-gateway`

Variable	Description	Example
`PORT`	Gateway listening port	`8080`
`KONG_ADMIN_URL`	Kong Admin API URL	`http://kong:8001`
`REDIS_URL`	Redis connection string	`redis://redis:6379`
`JWT_PUBLIC_KEY`	RS256 public key for JWT validation	`-----BEGIN PUBLIC KEY-----...`
`PROMPT_INJECTION_THRESHOLD`	Confidence threshold for blocking prompts	`0.85`
`RATE_LIMIT_DEFAULT`	Default requests per minute per tenant	`100`

B.2 `vessel-orchestrator`

Variable	Description	Example
`PORT`	Service listening port	`8081`
`DATABASE_URL`	PostgreSQL connection string	`postgresql://user:pass@postgres:5432/maars`
`KAFKA_BROKERS`	Kafka broker addresses	`kafka:9092`
`IDENTITY_SERVICE_URL`	`vessel-identity` gRPC endpoint	`grpc://vessel-identity:50051`
`MEMORY_SERVICE_URL`	`vessel-memory` gRPC endpoint	`grpc://vessel-memory:50052`
`SIMULATION_SERVICE_URL`	`vessel-simulation` gRPC endpoint	`grpc://vessel-simulation:50053`
`SWARM_SERVICE_URL`	`vessel-swarm` gRPC endpoint	`grpc://vessel-swarm:50054`
`SIMULATION_CONFIDENCE_THRESHOLD`	Min confidence to auto-approve	`0.80`
`MAX_TASK_DEPTH`	Maximum agent hierarchy depth	`5`

B.3 `vessel-llm-router`

Variable	Description	Example
`PORT`	Service listening port	`8082`
`REDIS_URL`	Redis for semantic cache	`redis://redis:6379`
`OPENAI_API_KEY`	Platform-level OpenAI key (fallback)	`sk-...`
`ANTHROPIC_API_KEY`	Platform-level Anthropic key (fallback)	`sk-ant-...`
`GOOGLE_API_KEY`	Platform-level Google key (fallback)	`AIza...`
`SEMANTIC_CACHE_THRESHOLD`	Cosine similarity threshold for cache hits	`0.95`
`DEFAULT_NANO_MODEL`	Default model for Nano tier	`llama-3-8b-instruct`
`DEFAULT_MID_MODEL`	Default model for Mid tier	`llama-3-70b-instruct`
`DEFAULT_FRONTIER_MODEL`	Default model for Frontier tier	`gpt-4.1`

B.4 `vessel-identity`

Variable	Description	Example
`PORT`	Service listening port	`8083`
`DATABASE_URL`	PostgreSQL connection string	`postgresql://user:pass@postgres:5432/maars`
`KAFKA_BROKERS`	Kafka broker addresses	`kafka:9092`
`JWT_PRIVATE_KEY`	RS256 private key for JWT signing	`-----BEGIN PRIVATE KEY-------...`
`DID_METHOD`	DID method for agent identity	`did:key`
`ED25519_PRIVATE_KEY`	Ed25519 private key for DID signing	`base64-encoded-key`
`ANS_REGISTRY_URL`	Agent Name Service registry URL	`https://ans.maars.ai`
`A2A_ENDPOINT`	Public A2A protocol endpoint	`https://a2a.maars.ai`
`JIT_TOKEN_TTL_SECONDS`	Default JIT token lifetime	`3600`
`CAEP_CHECK_INTERVAL_MS`	CAEP monitoring interval	`5000`

B.5 `vessel-memory`

Variable	Description	Example
`PORT`	Service listening port	`8084`
`QDRANT_URL`	Qdrant vector store URL	`http://qdrant:6333`
`NEO4J_URL`	Neo4j connection URL	`bolt://neo4j:7687`
`NEO4J_USER`	Neo4j username	`neo4j`
`NEO4J_PASSWORD`	Neo4j password	`password`
`EMBEDDING_MODEL`	Embedding model for vector generation	`text-embedding-3-large`
`OPENAI_API_KEY`	OpenAI key for embeddings	`sk-...`
`MEMORY_TTL_EPISODIC_DAYS`	Episodic memory expiry	`90`
`MEMORY_TTL_SEMANTIC_DAYS`	Semantic memory expiry	`365`

B.6 `vessel-sandbox`

Variable	Description	Example
`PORT`	Service listening port	`8085`
`DATABASE_URL`	PostgreSQL connection string	`postgresql://user:pass@postgres:5432/maars`
`S3_ENDPOINT`	S3/MinIO endpoint for artifacts	`http://minio:9000`
`S3_BUCKET`	Artifact storage bucket	`maars-artifacts`
`S3_ACCESS_KEY`	S3 access key	`minioadmin`
`S3_SECRET_KEY`	S3 secret key	`minioadmin`
`GUARDRAIL_SERVICE_URL`	`vessel-observability` gRPC endpoint	`grpc://vessel-observability:50055`
`ESCROW_SERVICE_URL`	`vessel-economics` gRPC endpoint	`grpc://vessel-economics:50056`
`WASM_MAX_MEMORY_MB`	Max WASM sandbox memory	`512`
`WASM_MAX_CPU_CORES`	Max WASM sandbox CPU cores	`2`
`WASM_MAX_EXECUTION_SECONDS`	Max WASM sandbox execution time	`300`

B.7 `vessel-economics`

Variable	Description	Example
`PORT`	Service listening port	`8086`
`DATABASE_URL`	PostgreSQL connection string	`postgresql://user:pass@postgres:5432/maars`
`KAFKA_BROKERS`	Kafka broker addresses	`kafka:9092`
`BLOCKCHAIN_PROVIDER`	Blockchain RPC endpoint	`https://mainnet.base.org`
`ESCROW_PRIVATE_KEY`	Platform escrow signing key	`0x...`
`ERC4337_FACTORY_ADDRESS`	Smart account factory contract	`0x...`
`X402_ENABLED`	Enable x402 payment protocol	`true`
`DEFAULT_LIABILITY_CAP_USD`	Default liability cap if not set	`1000`
`INSURANCE_API_KEY`	Insurance Telemetry API key	`ins_...`

B.8 `vessel-observability`

Variable	Description	Example
`PORT`	Service listening port	`8087`
`CLICKHOUSE_URL`	ClickHouse connection URL	`http://clickhouse:8123`
`KAFKA_BROKERS`	Kafka broker addresses	`kafka:9092`
`ARIZE_SPACE_ID`	Arize Phoenix space ID	`space_...`
`ARIZE_API_KEY`	Arize Phoenix API key	`ak_...`
`GUARDRAIL_MODEL`	LLM model for guardrail scoring	`gpt-4.1-mini`
`GUARDRAIL_THRESHOLD`	Min score to pass (0-1)	`0.75`
`IDENTITY_SERVICE_URL`	`vessel-identity` for token revocation	`grpc://vessel-identity:50051`

Appendix C: Kubernetes Deployment Reference

All MAARS services are deployed as Kubernetes Deployments with Horizontal Pod Autoscalers (HPA). The following table provides the recommended resource allocations and scaling parameters for each service.

Service	CPU Request	CPU Limit	Memory Request	Memory Limit	Min Replicas	Max Replicas	Scale Trigger
`vessel-gateway`	500m	2000m	256Mi	1Gi	3	20	CPU > 70%
`vessel-orchestrator`	1000m	4000m	512Mi	2Gi	2	10	CPU > 70%
`vessel-swarm`	500m	2000m	256Mi	1Gi	2	20	Active agents > 80% pool
`vessel-identity`	500m	2000m	256Mi	1Gi	3	15	CPU > 70%
`vessel-memory`	1000m	4000m	1Gi	4Gi	2	8	CPU > 60%
`vessel-llm-router`	500m	2000m	256Mi	1Gi	3	20	CPU > 70%
`vessel-sandbox`	2000m	8000m	2Gi	8Gi	2	50	Active sandboxes > 80%
`vessel-observability`	1000m	4000m	512Mi	2Gi	2	8	CPU > 70%
`vessel-simulation`	2000m	8000m	2Gi	8Gi	1	5	Active simulations > 80%
`vessel-research`	1000m	4000m	512Mi	2Gi	1	10	CPU > 70%
`vessel-economics`	500m	2000m	256Mi	1Gi	2	8	CPU > 70%
`vessel-interface`	500m	2000m	256Mi	1Gi	2	10	CPU > 70%

All services are deployed with:

- **Liveness Probe:** `GET /health` every 30 seconds, failure threshold 3.
- **Readiness Probe:** `GET /ready` every 10 seconds, failure threshold 3.
- **Pod Disruption Budget:** Minimum 1 available replica during rolling updates.

- **Network Policy:** Deny all ingress/egress by default; explicit allow rules for each service-to-service communication path.
- **Service Account:** Dedicated service account per service with minimal RBAC permissions.
- **Pod Security Context:** `runAsNonRoot: true`, `readOnlyRootFilesystem: true`, `allowPrivilegeEscalation: false`.

Appendix D: Glossary

Term	Definition
ABU	Autonomous Business Unit. An AI agent configured to operate as an independent business function within an enterprise.
A2A Protocol	Agent2Agent Protocol. An open standard (v1.0.0, Linux Foundation) for communication and interoperability between independent AI agent systems.
ABAC	Attribute-Based Access Control. A policy model that grants access based on attributes of the subject, resource, and environment.
Agent Card	A JSON metadata document published by an A2A server describing its identity, capabilities, skills, endpoint, and authentication requirements.
AIM	Agent Identity Management. The module responsible for append-only audit logging of all identity events.
ANS	Agent Name Service. A global registry of agents, analogous to DNS, that allows discovery by capability.
CAEP	Continuous Access Evaluation Protocol. A real-time token revocation mechanism that monitors agent behavior and revokes tokens on anomaly detection.
DAG	Directed Acyclic Graph. A graph data structure used to represent task dependencies in the orchestrator.
DePIN	Decentralized Physical Infrastructure Networks. Peer-to-peer networks of compute, storage, and connectivity nodes.
DID	Decentralized Identifier. A W3C standard for cryptographic, self-sovereign identifiers anchored to blockchains.
ERC-4337	An Ethereum standard for Smart Contract Accounts (programmable wallets) that support policy-based spending limits and multi-signature approval.
Frontier Agent	An agent running a top-tier LLM (GPT-4.1, Claude 3.5, Gemini 2.5 Pro) for complex reasoning tasks.
GoalPacket	The primary data structure representing a high-level task submitted by a human operator.
GraphRAG	Graph-based Retrieval-Augmented Generation. A technique combining vector search with knowledge graph traversal for highly contextual LLM responses.
Guardrail Agent	An isolated LLM judge that scores agent reasoning chains before execution to prevent silent failures and policy violations.

Term	Definition
JIT Provisioning	Just-In-Time Identity Provisioning. The practice of creating minimal, task-scoped credentials at the moment they are needed and destroying them immediately after.
MAARS	Master Autonomous Agentic Runtime System. The final name of the platform after Phase 4 completion.
Mid Agent	An agent running a mid-tier LLM (Llama 3 70B, Mistral 7B) for moderate complexity tasks.
Nano Agent	An agent running a small, fast LLM (Llama 3 8B) for simple, deterministic tasks.
NHI	Non-Human Identity. The identity of an AI agent or automated system, as distinct from a human user identity.
Right-Sizing	The process of assigning the appropriate cognitive tier (Nano, Mid, Frontier) to each sub-task based on its complexity and cost requirements.
RSI Engine	Recursive Self-Improvement Engine. The module that captures successful task completions and uses them as synthetic training data to continuously improve agent performance.
TaskGraph	The DAG of sub-tasks generated by the orchestrator for a GoalPacket.
VC	Verifiable Credential. A W3C standard for cryptographically verifiable digital credentials.
WASM	WebAssembly. A binary instruction format for a stack-based virtual machine, providing a memory-safe, sandboxed execution environment.
x402	A machine-to-machine payment protocol built on the HTTP 402 status code, enabling autonomous micro-transactions between agents.